



Final

Data



Algorithm

- ขั้นตอนการแก้ปัญหา
- เป็นลักษณะของแนวคิด ex. Pseudo code, Flowchart, Program

Algorithm Analysis

- สามารถบอกได้หลายวิธี
- สามารถเปรียบเทียบประสิทธิภาพได้
- พิจารณา : เวลา, พื้นที่หน่วยความจำ

Algorithm Efficiency

- number of process
- $f(n)$ = efficiency
- ถ้าเป็น Linear (No loop) $\Rightarrow$ efficiency = จำนวนคำสั่งทั้งหมด

Linear Loops

Ex. for (i=0; i<1000; i++) statement
end for
 $f(n) = 1000$

for (i=0; i<n; i++) statement
end for
 $f(n) = n$

for (i=0; i<1000; i+=2) statement
 $f(n) = 500$

for (i=0; i<n; i+=2) statement
 $f(n) = \frac{n}{2}$

Logarithmic Loops

* ถ้ามากกว่า 1 statement ในลูปจะเข้า
for (i=1; i<1000; i*=2) statement
 $f(n) = \lceil \log_2 1000 \rceil$

for (i=1; i<n; i*=2) statement
 $f(n) = \log_2 n$

for (i=1000; i>1; i/=2) statement
for (i=n; i>1; i/=2) statement

Nested Loops $\Rightarrow$ คูณนอก * คูณใน

for (i=0; i<10; i++)
for (j=1; j<10; j*=2) statement
end for
end for
 $f(n) = n \cdot \lceil \log_2 n \rceil$ ($10 \cdot \lceil \log_2 10 \rceil$)

for (i=0; i<10; i++)
for (j=0; j<10; j++) statement
end for
 $f(n) = n^2$ (10×10)

for (i=1; i<=10; i++)
for (j=0; j<=i; j++) statement
 $f(n) = \frac{n(n+1)}{2}$ ($1+2+3+\dots+10$)

Big-O $\Rightarrow$ ดูจากค่ามากที่สุด

- log n n n log n n^2 n^3 n^k 2^n $n!$
น้อย $\xrightarrow{\text{สนใจอันดับในชุดนี้มากที่สุด}} \rightarrow$ มาก

- big-O คือการวัดประสิทธิภาพ

Ex.
 $f(n) = 2n+3$ $f(n) = n^3+3n-2$ $f(n) = 1, f(n) = 1000$
 $f(n) = O(n)$ $f(n) = O(n^3)$ $f(n) = O(1)$

Ex. Algo Test():

code
code] 2
 $\frac{n}{5}$ - for (i=0; i<n; i+=5):
code
code] 2
for (j=n; j>1; j/=3):
code
code] 2
end for
code - 1
end for
code - 1
end test
 $f(n) = 3 + \frac{n}{5} (3 + 2 \log_3 n)$
 $O(n \log_3 n) = 3 + \frac{3n}{5} + \frac{2n \log_3 n}{5}$
 $= \frac{2n \log_3 n}{5} + \frac{3n}{5} + \frac{3}{5}$

- Add Square Matrices

```

Algorithm addMatrix (matrix1, matrix2, size, matrix3)
Add matrix1 to matrix2 and place results in matrix3
Pre  matrix1 and matrix2 have data
     size is number of columns or rows in matrix
Post matrices added--result in matrix3
1 loop (not end of row)
1  loop (not end of column)
1  add matrix1 and matrix2 cells
2  store sum in matrix3
2  end loop
2 end loop
end addMatrix
    
```

$n \times n$

$O(n^2)$

$f(n) = 2n^2$
 $\approx O(n^2)$

DATA STRUCTURES AND ALGORITHMS

- Multiply Square Matrices

```

Algorithm multiMatrix (matrix1, matrix2, size, matrix3)
Multiply matrix1 by matrix2 and place product in matrix3
Pre  matrix1 and matrix2 have data
     size is number of columns and rows in matrix
Post matrices multiplied--result in matrix3
1 loop (not end of row)
1  loop (not end of column)
1  loop (size of row times)
1  calculate sum of
    (all row cells) * (all column cells)
2  store sum in matrix3
2  end loop
2 end loop
3 return
end multiMatrix
    
```

$O(n^3)$

Algorithm test1(n)

```

for (i=0; i<n; i++)
    for (j=0; j<5; j++)
        app_code
        app_code
    end for
    app_code
end for
end test1
    
```

n

$5 \cdot 2$

5

app_code

app_code

2

$$f(n) = n((5 \cdot 2) + 1)$$

$$= 11n$$

$$O(n)$$

Algorithm test2(n)

```

for (i=1; i<n; i*=2)
    for (j=0; j<n; j+=2)
        app_code
    end for
    app_code
    app_code
end for
end test2
    
```

$\log_2 n$

$\frac{n}{2}$

app_code

$$f(n) = \log_2 n \left(2 + \frac{n}{2} \right)$$

$$= 2 \log_2 n + \frac{n \log_2 n}{2}$$

$$O(n \log_2 n)$$

Searching

Sequential Search

- ไล่เปรียบเทียบทีละตัวไปเรื่อยๆ จนกว่าจะเจอ
- ไล่แต่ตัวที่ไม่เจอแสดงว่าไม่มี

```

Algorithm seqSearch(list, last, target)
1  set looker to 0
2  loop looker < last AND target not equal list[looker]
1  increment looker
end loop
set loc to looker
if (target equal list[looker])
1  set found to loc
else
1  set found to false
end if
return found
End seqSearch1
    
```

```

in an Ordered List
Algorithm seqSearch2(list, last, target)
1  set looker to 0
2  loop looker < last AND loop is TRUE
1  if (target is equal to list[looker])
1  set loc to looker
2  set loop to FALSE
2  else
1  if (list[looker] > target)
1  set loop to FALSE
2  else
increment looker
3  end loop
4  return loc
End seqSearch2
    
```

3 กรณี

- Best case $\rightarrow$ เจอครั้งแรก $\rightarrow$ เปรียบ 1 ครั้ง $\rightarrow O(1)$
- Worst case $\rightarrow$ เปรียบถึง last $\rightarrow$ ทุกครั้ง (ทุกตัว) $\rightarrow O(n)$
- Average case $\rightarrow$ เปรียบโดยหาคะแนนการเปรียบเทียบหารจำนวน $\rightarrow (n+1)/2 \rightarrow O(n)$

Binary Search $\Rightarrow$ Only in order

- เปรียบเทียบข้อมูลกลาง
- ไล่เท่าที่ไล่เรื่อยๆ จนกว่าจะเจอ
- สูตรกลาง $\lfloor (begin + end) / 2 \rfloor$

```

Algorithm binarySearch(list, last, target)
1  set begin to 0
2  set end to last
3  loop (begin <= end)
1  set mid to (begin+end)/2
2  if (target > list[mid])
1  set begin to mid+1
3  else if (target < list[mid])
1  set end to mid-1
4  else
1  set begin to (end+1)
5  end if
4  end loop
set loc to mid
if (target is equal to list[mid])
1  set found to TRUE
7  else
1  set found to FALSE
8  end if
9  return found
End binarySearch
    
```

- Best case $\rightarrow$ เปรียบ 1 ครั้งกลาง $\rightarrow O(1)$

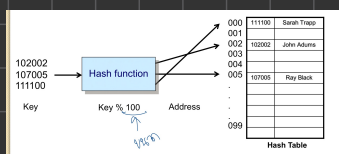
- Average case $\rightarrow$ เปรียบ $\rightarrow (2^0 \times 1 + 2^1 \times 2 + 2^2 \times 3 + \dots + 2^{(m-1)} \times m) / n \rightarrow O(\log_2 n)$

$m =$ ระดับ n ใน level m (ความสูง)

- Worst case $\rightarrow$ หาจนเจอคือตัวเดียว $\rightarrow O(\log_2 n)$

Hashing $\Rightarrow$ key, value $\rightarrow O(1) \rightarrow$ จัดข้อมูลเข้า key $\rightarrow$ ใช้ Hash function หา Location data ที่เก็บ Hash table

- ใช้ค่า $key \% \text{ขนาด Hash table}$



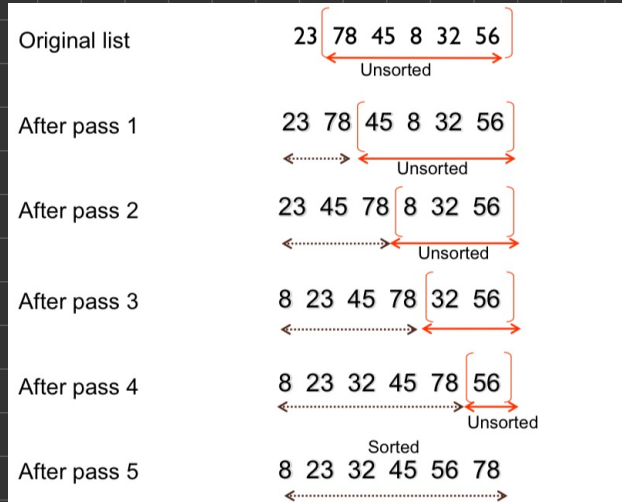
ปัญหา $\Rightarrow$ key ไล่เรื่อยๆ จนเจอ hash ที่ซ้ำเกิดซ้ำๆ กัน $\Rightarrow$ ถ้าหาเจอในขั้น Rehashing

- Linear Probing $\Rightarrow$ นำ key ผ่าน hash function ไล่จนกว่าจะเจอ โดยช่วง $=$ ไล่ $\Rightarrow$ นำค่าที่เจอมาใส่ที่ว่าง (Lo has + รหัส) ของขนาด $\rightarrow$ จัดข้อมูลจนจบ Search เสร็จ
- Buckets $\Rightarrow$ อยู่ตำแหน่งเดียวได้ ถ้าพบค่าเหมือนกัน $\Rightarrow$ ใช้แบบ Array 2 มิติ $\Rightarrow$ หาก 5 มีค่าเท่ากับ 6 แสดงว่ามี 5
- Chaining $\Rightarrow$ เก็บค่าไว้เหมือนกัน โดยไม่จำกัดจำนวน $\rightarrow$ ใช้แบบ Link List

Sorting $\Rightarrow$ เรียงลำดับข้อมูล

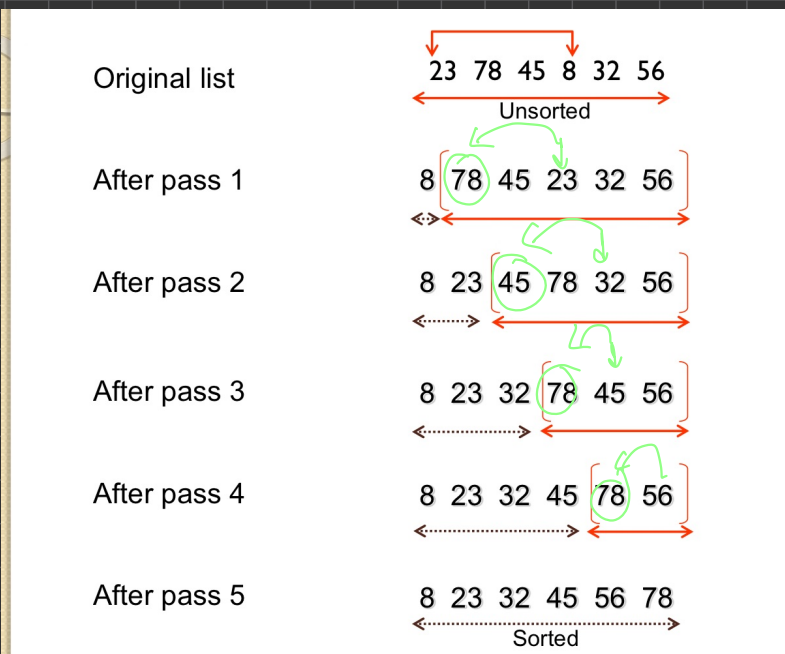
- $O(n^2) \Rightarrow$ insertion, selection, bubble
- $O(n \log n) \Rightarrow$ merge, quick

Insertion Sort $\Rightarrow$ นำข้อมูลตัวแรกของ list ย้ายไปยัง list ใหม่ที่เรียงไว้ก่อนหน้าจนกว่าจะพบตำแหน่งที่เหมาะสม



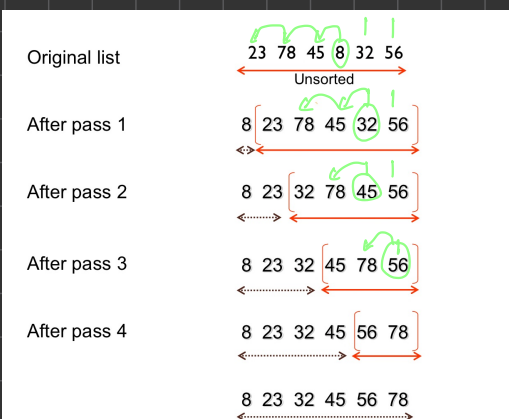
- ต้องเปรียบเทียบข้อมูลทั้งหมด $n-1$ ตัว
- เปรียบเทียบเสร็จก็ต้องไปหาตำแหน่งที่ควรใส่ (ที่ว่างแล้ว) (กรณีเฉลี่ย $n/2$)
- $f(n) = (n-1)(n/2)$
- $O(n^2)$

Selection Sort $\Rightarrow$ เลือกค่าที่น้อยที่สุดจาก Unsort list แล้วสลับข้อมูลกับตัวแรกของ Unsort list แล้วนำค่าที่น้อยที่สุดไปใส่ใน Sorted list



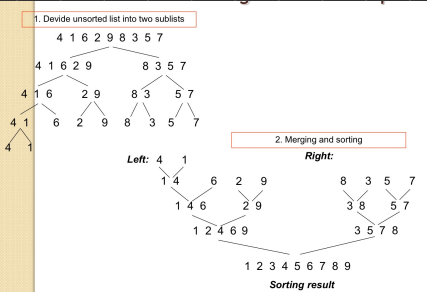
- รอบแรก n รอบ $n-1, n-2, \dots$
- สลับที่ 1 ครั้ง
- $f(n) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2$
- $O(n^2)$

Bubble Sort $\Rightarrow$ ถ้าหากตัวสุดท้ายที่พบแล้วมันน้อยกว่าตัวก่อนหน้าก็สลับที่กัน ถ้าหากว่าค่าที่พบมากกว่าตัวก่อนหน้าก็ไม่ต้องทำอะไร



- $f(n) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2$
- $O(n^2)$

Merge sort → แบ่ง เป็น 2 ส่วน เท่าๆกัน จนไม่สามารถแบ่งได้อีก (ใบเดียวตัว) วิธีนี้เหมาะกับ Sublist แล้วจึงมารวมกันใหม่



Total times = $\sum_{i=1}^N (\text{Number of lists in level } i) \times (\text{Number of comparisons in level } i)$

$$= 1 \times (n-1) + 2 \times ((n/2)-1) + 4 \times ((n/4)-1) + \dots + (2-1) \times (n/2)$$

$$= n-1 + n-2 + n-4 + \dots + n - n/2$$

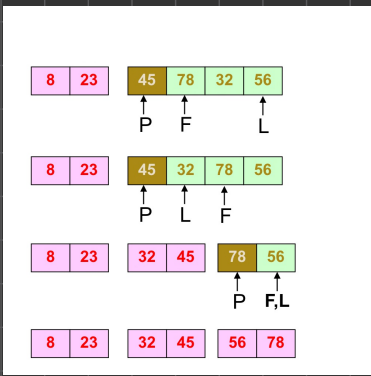
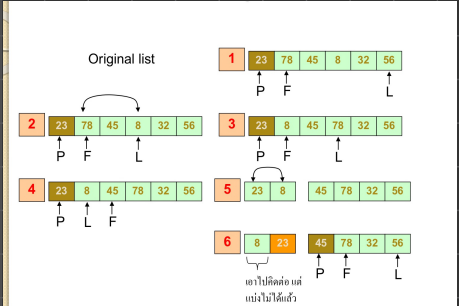
$$= n + n + n + \dots + n - (1+2+4+8+n/2)$$

n times = level of tree

$$= n \log_2 n - (n-1)$$

The merge sort efficiency is $O(n \log_2 n)$

Quick Sort → แบ่งรอบ pivot → ใช้ first / last ของ array โดย first จะจัดไปจาก pivot → ถ้า $\text{pivot} \leq \text{first}$ ให้ย้าย first ไปทางขวา ถ้า $\text{pivot} > \text{last}$ ให้ย้าย last ไปทางซ้าย



- if pivot แบ่ง 2 ชุด เท่ากัน $\Rightarrow$ ดีสุด $O(n \log_2 n)$
- แบ่งเกือบเร็ว $\Rightarrow O(n^2)$

Greedy → คำนวณโลภ → หาผลสัมฤทธิ์ที่ดีที่สุด ณ ขณะนั้น → ไม่รับประกันว่าจะดีที่สุด

Ex. Coin exchange $\Rightarrow$ มี 10, 5, 2, 1 บาท โดยใน 1 บาทมีเหรียญน้อยที่สุด $\Rightarrow$ ใช้เหรียญที่มากที่สุด $\Rightarrow$ ถ้า greedy จะใช้ 10 = 1, 2 = 1, 1 = 1 ใช้ 3 แต่สามารถใช้ 5, 5 จึงดีที่สุด

Classroom $\Rightarrow$ แบ่งห้อง โดยนักเรียนไม่สนใจ

Knapsack $\Rightarrow$ ขนของแพงๆ ใส่ในกระเป๋าไม่เต็ม

Set-covering $\Rightarrow$ ถ้าหาทั้งหมดคือ $O(2^n)$ n เป็นจำนวนวิธี
 $\Rightarrow$ Greedy $\Rightarrow$ เลือกความยากก่อน อาจจะไม่ได้ออก $O(n^2)$
ผลลัพธ์ที่ได้จะประมาณค่าเรียก Approximation algorithm

Travelling salesman $\Rightarrow$ ทำอย่างไรไปเยือนทุกเมือง โดยเสียเวลาน้อยที่สุด
 $\Rightarrow$ แก้ด้วย กราฟค้นหาทางสั้นสุด ใช้ $O(n^3)$
 $\Rightarrow$ แต่ถ้าใช้ Greedy $\Rightarrow$ ถ้าพบจุดที่เริ่ม $\Rightarrow$ เลือกเส้นทางที่ยังไม่เคยไปจึงได้ผลลัพธ์

Recursion → การทำซ้ำ โดยการใช้ตัวเรียกซ้ำ

2 ส่วน → base case, general case ex. Base: fac(0)

Factorial

Sen: $n! = fac(n-1)$

- ไม่ควรใช้เพื่อ โครงสร้างไม่เหมาะสม, เรียกซ้ำ ทำซ้ำมากเกินไป, ใช้เพื่อ / ธรรมดา

Ex. Recursive

gcd(a, b) = $\begin{cases} a & \text{if } b = 0 \\ b & \text{if } a = 0 \\ \text{gcd}(b, a \bmod b) & \text{otherwise} \end{cases}$

if b = 0
if a = 0
otherwise

gcd(10, 5) = gcd(5, 10 % 5)
= gcd(5, 0)
= 5

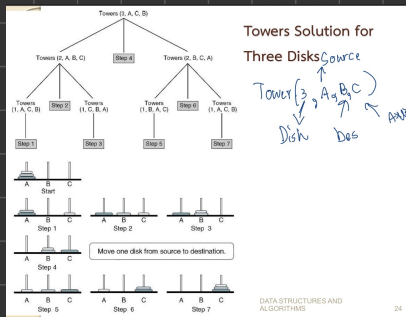
Fibonacci Numbers

0 1 1 2 3 5 8 13 21

fac(n) fac(n-1) fac(n-2)

Fibonacci(n) = $\begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) & \text{otherwise} \end{cases}$

f(4) = f(3) + f(2)
= (f(2) + f(1)) + (f(1) + f(0))
= (1 + 1) + 1 = 3



Divide and Conquer

- Divide → แบ่งปัญหาเป็นส่วนๆ
- Conquer → แก้ปัญหาย่อย (ทำเป็น Recursion)
- Combine → รวมผลลัพธ์

Ex.

Merge Sort → Divide → แบ่งเป็น 2 ส่วนเท่าๆ กัน จนแบ่งไม่ได้อีก
→ Conquer → เรียงย่อย
→ Combine → รวม sublist ที่เรียงจนเป็น list เดียว

Quick Sort → Divide → แบ่งย่อย 2 ชุดด้วย Pivot
→ Conquer → เรียงย่อยตาม Partition
→ Combine → เรียงจาก Pivot ของ list ย่อย

Modular Exponentiation → $(a^b) \bmod m = [(a \bmod m)^b] \bmod m$

Ex. $(3^5) \bmod 4 = [(3 \bmod 4)^5] \bmod 4$
= $(3^5) \bmod 4 = 3$

$2^{10} \bmod 10 \rightarrow$ Brute Force → คำนวณด้วย mod → $O(n)$

→ Divide and Conquer → $[(2^{10} \bmod 10)^2] \bmod 10$
= $(2^{10} \bmod 10)^2 \bmod 10$

↓
 $2^2 \bmod 10 = 2^2 (2^0 \bmod 10)^2 \bmod 10$
 $2^0 \bmod 10 = 1$

→ $O(\log n)$